WSABI Summer Research Lab **Summer 2026**

Lecture 2: Multivariable Linear Regression

Instructor: Jonathan Pipping-Gamón *Author: RB, JP*

2.1 Example: NCAA Men's Basketball Power Ratings

We have a dataset of game results from the 2022-2023 NCAA men's basketball season. As pictured in Figure 2.1, each row represents a game and includes the following information:

- i: index of the i^{th} game
- $H(i)$: index of the home team
- $A(i)$: index of the away team
- y_i: the score differential of game i (home team score - away team score)

i	$H(i)$	$A(i)$	y_i
1	Abilene Chr	Jackson St	9
2	Akron	S Dakota St	1
3	Alabama	Longwood	21
4	Arizona	Nicholls St	42
5	Arizona St	Tarleton St	3

Table 2.1: Head of the 2022-2023 NCAA men's basketball dataset

We intend to use this data to generate power ratings (team strength) for each team, which we can do by setting up a model for the score differential.

2.1.1 Setting Up the Model

Suppose each team j has a latent (unobserved) power rating β_j. Then, we can model the outcome (score differential) of the i^{th} game as follows:

$$y_i = \beta_0 + \beta_{H(i)} - \beta_{A(i)} + \epsilon_i$$

where $\beta_{H(i)}$ and $\beta_{A(i)}$ are unknown constants representing the strength of the home and away teams, and ϵ_i is a mean-zero noise term ($\mathbb{E}[\epsilon_i] = 0$). What does β_0 represent?

What does this look like in each line of the dataset?

$$\begin{aligned} y_1 &= \beta_0 + \beta_{DePaul} - \beta_{Loyola} + \epsilon_1 \\ y_2 &= \beta_0 + \beta_{Duke} - \beta_{Jacksonville} + \epsilon_2 \\ y_3 &= \beta_0 + \beta_{Miami} - \beta_{Evansville} + \epsilon_3 \\ &\vdots \end{aligned}$$

In matrix-vector form, we can write this as

$$\begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \end{bmatrix} = \begin{bmatrix} 1 & 1 & -1 & 0 & 0 & 0 & 0 & \dots \\ 1 & 0 & 0 & 1 & -1 & 0 & 0 & \dots \\ 1 & 0 & 0 & 0 & 0 & 1 & -1 & \dots \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_{DePaul} \\ \beta_{Loyola} \\ \beta_{Duke} \\ \beta_{Jacksonville} \\ \beta_{Miami} \\ \beta_{Evansville} \\ \vdots \end{bmatrix} + \begin{bmatrix} \epsilon_1 \\ \epsilon_2 \\ \epsilon_3 \\ \vdots \end{bmatrix}$$

We can write this more compactly as

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\epsilon}$$

where $\mathbf{y}$ is a vector of score differentials, $\mathbf{X}$ is a scheduling matrix, $\boldsymbol{\beta}$ is a vector of unknown parameters, and $\boldsymbol{\epsilon}$ is a vector of noise terms. Note that $\mathbf{X}$ consists of an intercept column and a one-hot encoding of the home (1) and away (-1) teams in the remaining columns.

Now, how do we estimate the unknown team strength parameters $\boldsymbol{\beta}$ from the observed data $(\mathbf{X}, \mathbf{y})$?

2.1.2 Estimating Model Parameters

Recall that in Lecture 1 we estimated (β_0, β_1) by minimizing the **Residual Sum of Squares**. Similarly, in multi-variable regression, we minimize the RSS. The optimization problem is:

$$\begin{aligned} \widehat{\boldsymbol{\beta}} &= \arg\min_{\boldsymbol{\beta}} \mathrm{RSS}(\boldsymbol{\beta}) \\ &= \arg\min_{\boldsymbol{\beta}} \sum_{i=1}^{n} (y_i - x_i^T \boldsymbol{\beta})^2 \end{aligned}$$

where x_i is the i^{th} row of $\mathbf{X}$ and $x_i^T\boldsymbol{\beta} = x_i \cdot \boldsymbol{\beta} = x_{i1}\beta_0 + x_{i2}\beta_1 + \ldots + x_{i(j+1)}\beta_j$ is the dot product.

We can solve this with calculus, setting the gradient (the multivariable analog of the derivative) equal to 0 and solving for $\boldsymbol{\beta}$ to obtain our estimate $\widehat{\boldsymbol{\beta}}$. Our solution to this problem is:

$$\widehat{\boldsymbol{\beta}} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y} \tag{2.1}$$

when $\mathbf{X}$ has full column rank. These are called the **Normal Equations** for linear regression.

2.1.3 What is Regression Doing Geometrically?

The fitted values

$$\widehat{\mathbf{y}} = \mathbf{X}\widehat{\boldsymbol{\beta}}$$

are the model's best approximation to $\mathbf{y}$ using linear combinations of the columns of $\mathbf{X}$. In geometric terms, linear regression projects $\mathbf{y}$ onto the **column space** of $\mathbf{X}$. The residual vector

$$\mathbf{r} = \mathbf{y} - \widehat{\mathbf{y}}$$

is the part of the outcome that the model could not explain.

This perspective is useful for all three examples in this lecture:

- In the NCAA ratings model, fitted score differentials must come from home-court advantage plus team-strength differences.
- In the punting example, adding X_i^2 expands the set of curves the model can fit.
- In the draft example, adding spline basis functions expands that space even further, which gives more flexibility but also more opportunity to overfit.

There is also a small wrinkle in the NCAA setup: team ratings are only identifiable up to an additive constant. If we add the same number to every team's rating, every difference $\beta_{H(i)} - \beta_{A(i)}$ stays the same. So $\mathbf{X}^T\mathbf{X}$ is singular in this example, and software must impose a convention such as centering the ratings, dropping one team column, or using a generalized inverse. The home-court advantage remains identifiable, but the absolute level of the ratings does not.

2.1.4 Estimating Power Ratings

Now that we know how to estimate $\widehat{\boldsymbol{\beta}}$, we can use `R` to fit our regression model. In the code below, `X` is the full design matrix, and its first column is already the all-ones column corresponding to home-court advantage. So we include `+ 0` to tell `lm()` not to add a second intercept column automatically.

```
# fit the linear model
model = lm(score_diff ~ X + 0, data = ncaa_data)
# display model coefficients
model
```

Our intercept $\widehat{\beta}_0$ is the average score differential after adjusting for relative team strength, so it represents the home-court advantage. In our model, $\widehat{\beta}_0 = 2$ means that being the home team is associated with roughly a 2-point scoring advantage.

We plot the sorted power ratings for each team in Figure 2.1. A zoom-in on 25 randomly-selected teams is shown in Figure 2.2, now with 95% confidence intervals. These intervals depend on the centering convention used to make the ratings identifiable. For comparing two teams, the most meaningful uncertainty is the uncertainty in the difference between their ratings. If we wanted to predict the score differential for a game between two teams, we would simply take the difference between their power ratings and add the home-court advantage.

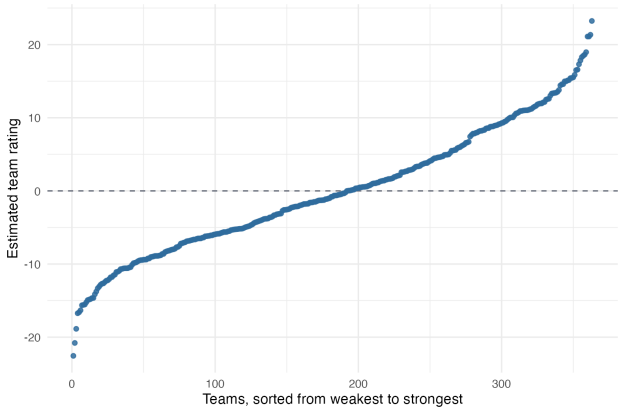

Figure 2.1: Team power ratings for the 2022-2023 NCAA men's basketball season

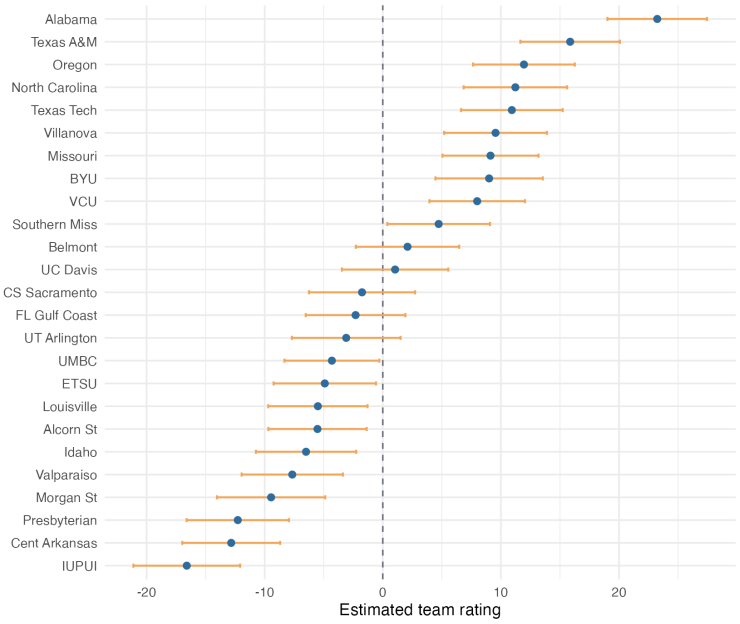

Figure 2.2: Zoom-in on 25 randomly-selected teams in the 2022-2023 NCAA men's basketball season

2.2 Example: Net Punt Position in American Football

We have a dataset of NFL punts. Each row represents a punt and includes the following information:

- i: index of the i^{th} punt
- X_i: yard line before the punt (yards from the opponent's end zone)
- y_i: the opponent's yard line after the punt

We model y_i as a function of X_i. Mathematically, we express this as

$$y_i = f(X_i) + \epsilon_i$$

where f is some unknown function and ϵ_i is a mean-zero noise term. As before, we want to estimate the expected value of this quantity:

$$\mathbb{E}[y_i \mid X_i] = f(X_i)$$

Generally, it's a good idea to start with some exploratory data analysis. We plot the relationship between pre-punt field position (X_i) and post-punt field position (y_i) in Figure 2.3.

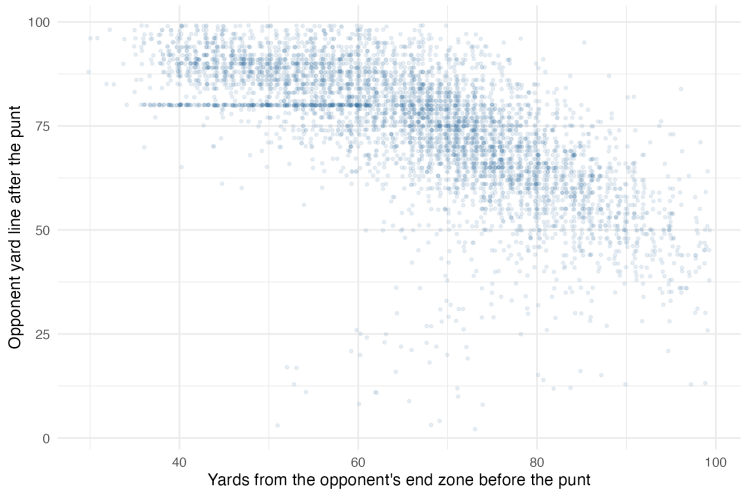

Figure 2.3: Relationship between pre-punt and post-punt field position

As highlighted in Figure 2.4, we see that the relationship between starting field position and the opponent's yard line after the punt bends downward near the opponent's end zone. Long-field punts behave differently from short-field punts because touchbacks become more likely. How can we use linear regression to capture that nonlinear relationship?

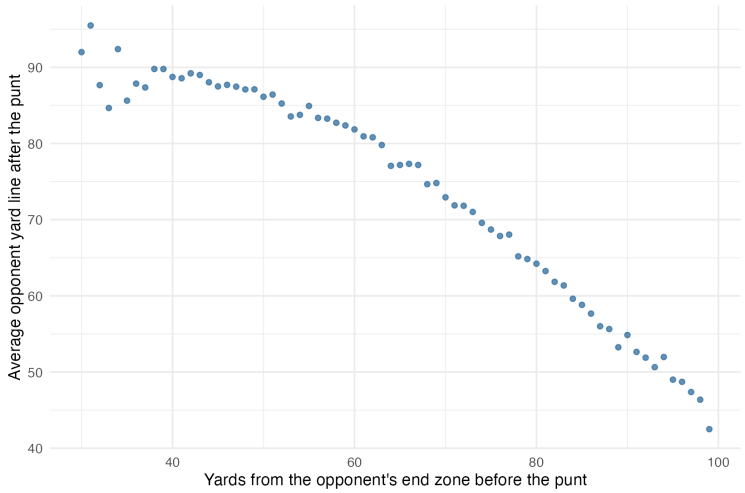

Figure 2.4: Average post-punt field position by starting field position

2.3 Data Transformations

We can use a **data transformation** to capture nonlinear relationships. For example, we can transform the yard line X_i to X_i^2 to capture the curvature we noted in Figure 2.4.

If our linear model setup would be

$$y_i = \beta_0 + \beta_1 X_i + \epsilon_i, \quad \mathbb{E}[\epsilon_i] = 0$$

then our transformed **quadratic model** is

$$y_i = \beta_0 + \beta_1 X_i + \beta_2 X_i^2 + \epsilon_i, \quad \mathbb{E}[\epsilon_i] = 0$$

In matrix-vector form, we maintain the form

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\epsilon}$$

but now $\mathbf{X}$ is a matrix with three columns instead of two: the intercept column, the X_i column, and the X_i^2 column. Explicitly, we have

$$\underbrace{\begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \end{bmatrix}}_{\mathbf{y}} = \underbrace{\begin{bmatrix} 1 & X_1 & X_1^2 \\ 1 & X_2 & X_2^2 \\ 1 & X_3 & X_3^2 \\ \vdots & \vdots & \vdots \end{bmatrix}}_{\mathbf{X}} \underbrace{\begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{bmatrix}}_{\boldsymbol{\beta}} + \underbrace{\begin{bmatrix} \epsilon_1 \\ \epsilon_2 \\ \epsilon_3 \\ \vdots \end{bmatrix}}_{\boldsymbol{\epsilon}}$$

The model is still linear in the coefficients. What changed is the set of columns we gave the model. In Lecture 1, the fitted-value vectors corresponded to straight-line functions evaluated at the observed X_i's. After adding X_i^2, the fitted-value vectors correspond to quadratic functions evaluated at those same points. Least squares still does the same thing: project $\mathbf{y}$ onto the span of the columns of $\mathbf{X}$.

As before, we solve for $\widehat{\boldsymbol{\beta}}$ with the normal equations from Equation 2.1:

$$\widehat{\boldsymbol{\beta}} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

We turn to `R` to fit our linear and quadratic models.

```
# linear model
l_model = lm(next_yardline ~ yds_to_end, data = punt_data)
l_coef = l_model$coefficients
# quadratic model
q_model = lm(next_yardline ~ yds_to_end + I(yds_to_end^2), data = punt_data)
q_coef = q_model$coefficients
```

We plot the models in Figure 2.5, and the quadratic model captures the short-field curvature better than the straight-line fit.

Figure 2.5: Linear and quadratic models fit to the data

2.4 Example: NFL Draft Expected Value Curve

We have a dataset of NFL draft picks. As shown in Table 2.2, each row represents a drafted non-quarterback from 2011 through 2020 and contains the following information:

- i: index of the i^{th} draft pick
- X_i: player i's draft pick number
- y_i: player i's second-contract annual value as a percentage of the salary cap, used as a public proxy for on-field value (Baldwin, 2025)

Player	Pos.	Draft Year	Draft Pick	Value (% cap)
Eric Fisher	LT	2013	1	7.7
Jadeveon Clowney	ED	2014	1	8.5
Myles Garrett	ED	2017	1	12.6
Von Miller	ED	2011	2	12.3
Luke Joeckel	LG	2013	2	4.8

Table 2.2: Head of the NFL draft dataset

We model the outcome of a draft pick y_i as a function of draft position X_i. Mathematically, we have

$$y_i = f(X_i) + \epsilon_i$$

where f is some unknown function and ϵ_i is a mean-zero noise term. As before, we want to estimate the expected value of this quantity:

$$\mathbb{E}[y_i \mid X_i] = f(X_i)$$

We proceed by plotting the relationship between draft position and this public value proxy in Figure 2.6.

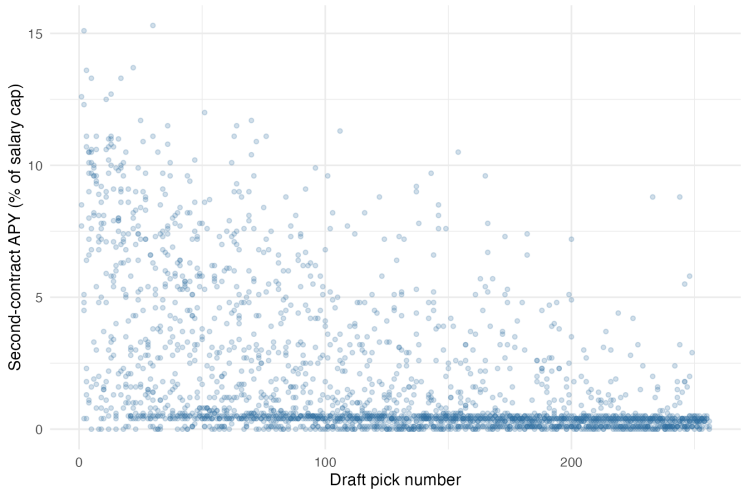

Figure 2.6: Relationship between draft position and second-contract annual value

Once again, the expected value $\mathbb{E}(y|X)$ looks nonlinear. In particular, the relationship is convex: the dropoff between picks t and $t+1$ decreases as t increases. How do we select a function to model this?

2.4.1 Splines

To model a general nonlinear relationship, we can use a piecewise polynomial function, or **spline**. To implement this, we fit a separate (usually-cubic) polynomial to different subsections of the data. For example, we could fit a separate cubic polynomial in each round of the draft. Since there are 32 picks per round, our 7 separators (or **knots**) would be at 33, 65, 97, 129, 161, 193, and 225.

To force our fitted spline to be **smooth**, we enforce second-order (or C2) continuity, which means that at each knot, the spline, its first derivative, and its second derivative are all continuous.

2.4.2 Basis-Function View of a One-Knot Spline

Suppose we fit a cubic spline with one knot at $x = k$ (for example, $x = 129$, near the middle of the draft). Instead of thinking about two separate cubic equations, it is cleaner to think in terms of **basis functions**. We build a design matrix whose columns are smooth transformations of x, such as

$$1, \quad x, \quad x^2, \quad x^3, \quad (x-k)_+^3$$

where

$$(x-k)_+^3 = \begin{cases} 0, & x \le k \\ (x-k)^3, & x > k. \end{cases}$$

Then our spline model is still just a linear regression:

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \beta_3 x_i^3 + \beta_4 (x_i - k)_+^3 + \epsilon_i.$$

The continuity conditions are built into the basis itself, so we do not need to solve for separate left-side and right-side coefficients by hand. This is the key idea to remember: a spline is still linear in the coefficients, but it uses a richer set of basis columns.

R's `bs()` function uses a numerically stable B-spline basis rather than exactly these truncated-power columns, but the idea is the same: we are adding spline basis columns to the design matrix.

2.4.3 Fitting the Full Spline

Now we fit the full 7-knot cubic spline on the NFL draft data. We use `R` to fit the model.

```
# fit the full spline
full_spline = lm(performance_value ~ splines::bs(draft_pos, degree = 3,
            knots = seq(33, 225, by = 32)), data = draft_data)
# get coefficients
full_coef = full_spline$coefficients
```

However, when we plot the model in Figure 2.7, we notice that our curve is a bit wiggly late in the draft. This suggests that the model may be too flexible: the column space is large enough that the fitted curve can start chasing noise. To fix this, we can reduce the number of knots to get a smoother fit.

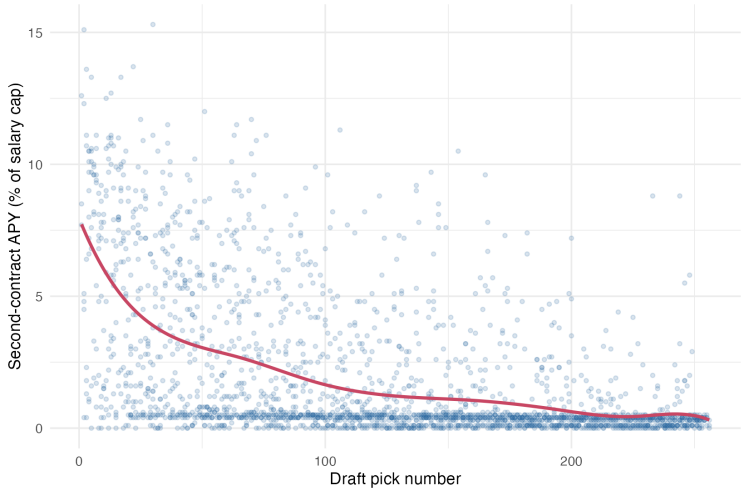

Figure 2.7: Full cubic spline fit to the data.

The degrees of freedom control the complexity of the spline fit. More knots and higher-degree polynomials create more basis columns, which gives the model more flexibility. In `R`, the `df` argument in `bs()` directly controls the number of spline basis columns returned by the function. Here, fewer degrees of freedom means fewer basis columns and a smoother fit. Let's try re-fitting the model with 5 degrees of freedom.

```
# fit reduced spline with 5 df
red_spline = lm(performance_value ~ splines::bs(draft_pos, degree = 3,
           df = 5), data = draft_data)
# get coefficients
red_coef = red_spline$coefficients
```

When we plot this new spline in Figure 2.8, it's clear we have a much smoother fit that is less prone to chasing noise.

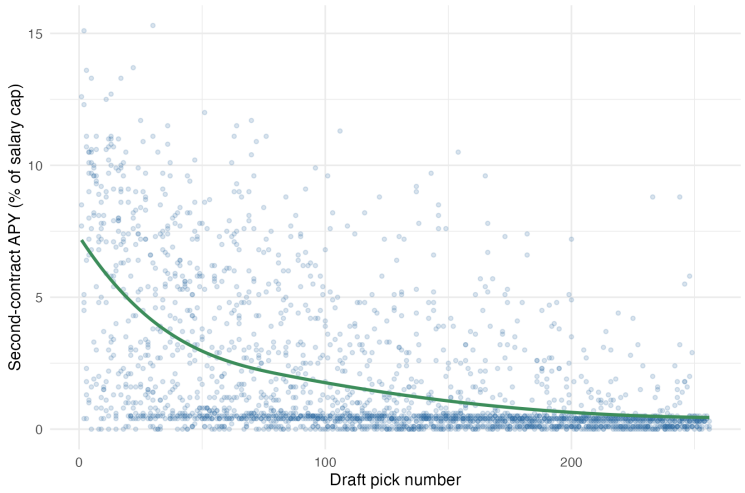

Figure 2.8: Reduced cubic spline fit to the data.

Takeaways

- **Multivariable regression is still least squares.** Even when the model has many predictors, the goal is the same: choose coefficients so that the fitted values

$$\hat{y} = X\hat{\beta}$$

  are as close as possible to the observed outcomes y.

- **Geometrically, regression is projection.** The columns of X define the set of fitted-value vectors the model is allowed to produce. Least squares projects y onto this column space, and the residuals

$$r = y - \hat{y}$$

  are the part of the outcome that the model does not explain.

- **The design matrix is the model.** In the NCAA ratings example, the columns of X encode home-court advantage and team-strength differences. In the punting example, adding X_i^2 lets the model capture curvature. In the draft example, spline basis columns let the model fit a more flexible nonlinear pattern.

- **Some coefficients are only meaningful after choosing a convention.** In sports-rating models, team ratings are relative: adding the same constant to every team's rating does not change any predicted score differential. This means the absolute level of the ratings is not identifiable, so we need a constraint or convention such as centering the ratings.

- **Uncertainty matters.** Regression estimates are not ground truth. Confidence intervals remind us that fitted coefficients and ratings are estimated from noisy data. For ratings, uncertainty in differences between teams is often more meaningful than uncertainty in an individual team's absolute rating.

- **Linear regression can model nonlinear relationships.** "Linear" means linear in the coefficients, not necessarily linear in the original variable. By adding transformed columns such as X_i^2, $(X_i - k)_+^3$, or spline basis functions, we can fit nonlinear curves using the same least-squares machinery.

- **Flexibility is a tradeoff.** More basis columns give the model a larger column space and more ability to match the data. But too much flexibility can make the fitted curve chase noise rather than signal, so smoother or lower-dimensional models can sometimes be more useful.

References

[Baldwin] Baldwin, B., *NFL Draft Value Chart*, Open Source Football, 2025.